PRODUCT REQUIREMENTS DOCUMENT

Distributed Real-Time Drawing Board

with Mini-RAFT Consensus

Cloud Computing — Team Project

TEAM MEMBERS

Shrish PES2UG23AM097 | **Saffiya PES2UG23AM87**

Rushad PES2UG23AM061 | **Ayesha PES2UG23AM921**

Version 1.0 | 3-Week Sprint | Go + gRPC + React + Docker

1. Executive Summary

This document is the single source of truth for the Mini-RAFT Distributed Drawing Board project. It defines every component, every API, every data model, every protocol rule, every Docker configuration, every team member responsibility, and every week-by-week checkpoint. No implementation decision should be made without reference to this document, and any deviation must be discussed and documented.

The system is a fault-tolerant, real-time collaborative drawing board backed by a simplified RAFT consensus protocol. Three Go replica nodes form a cluster that maintains an append-only stroke log with majority-quorum commits. A Gateway service handles all WebSocket clients and routes strokes to the current leader. A React + Vanilla JS frontend provides a Figma-lite drawing experience with an embedded live cluster state visualizer, per-user color identity, undo/redo via log compensation, and a Chaos Mode button for live failure injection.

Why this will stand out

One-third of the class has the same spec. Most teams will build a Node.js system with fake RAFT, a plain canvas, and no observability. This team builds: correct gRPC-based RAFT with write-ahead log persistence, a live cluster dashboard that makes elections visible in real time, per-user colour identity, log-compensation undo, a Prometheus metrics endpoint on every replica, a terminal TUI, bind-mounted WAL logs, and a Chaos Mode that randomly picks between graceful and hard kills. The demo video will be objectively more impressive.

2. Goals & Non-Goals

2.1 Goals

- **Implement a correct, simplified RAFT consensus protocol in Go across 3 containerised replicas.**
  - Leader election with random timeouts (500–800ms), majority voting, split-vote retry
  - Log replication with prevLogIndex/prevLogTerm safety check
  - Commit once majority ACK; committed entries never overwritten
  - Catch-up sync for restarted nodes via /sync-log delta protocol
- Persist the stroke log to a write-ahead log (WAL) on disk so a restarted container retains committed entries.
- Build a WebSocket Gateway in Go that routes client strokes to the leader and handles leader failover without disconnecting clients.
- Build a Figma-lite frontend: smooth canvas, per-user colour identity, colour/width tools, eraser, undo/redo, and a live embedded cluster state panel.
- Implement Chaos Mode: a button that calls a Gateway endpoint to randomly docker stop or docker kill a replica, triggering a live election visible in the dashboard.
- Full observability: structured zap JSON logs to stdout, log files on bind-mounted volumes, Prometheus /metrics endpoints on every replica and gateway, and an optional terminal TUI.
- Zero-downtime hot-reload: editing any replica source file triggers air auto-reload; the cluster must hold an election and resume serving without dropping client connections.
- Bonus — Undo/Redo: personal undo appends a compensation entry to the RAFT log; undo is client-side scoped but the compensation event goes through the full consensus pipeline.

2.2 Non-Goals

- Full production-grade RAFT (log compaction, snapshots, pre-vote, joint consensus, membership changes) — out of scope.
- Persistent WebSocket sessions across server restarts — clients reconnect automatically.
- Authentication or user accounts — colour identity is session-scoped only.
- Cloud deployment (AWS/GCP) — optional bonus, not required for full marks.
- More than 3 replicas in the core implementation — 4th replica is a bonus.
- Cross-user collaborative undo — undo only affects the current user's own strokes.

3. Team & Component Ownership

Grading is individual (component ownership + viva). Each person owns exactly one subsystem. Shared interfaces are locked down in Week 1 so all four can develop in parallel from Week 2 onward. Every person must understand the full system for viva, but must have deep expertise in their own component.

Shrish (PES2UG23AM097) — RAFT Core Engine
State machine, election timer, vote logic, term management, heartbeat sender — the brain of the cluster.

Saffiya (PES2UG23AM87) — Log Replication + WAL + Catch-Up Protocol

AppendEntries RPC, prevLogIndex safety, commit logic, write-ahead log, /sync-log delta sync — data safety.

Rushad (PES2UG23AM061) — Gateway + Docker Infrastructure + Chaos Mode
WebSocket server, leader routing, failover handling, docker-compose, air hot-reload, Chaos Mode endpoint.

Ayesha (PES2UG23AM921) — Frontend + Live Dashboard + Undo/Redo
React UI chrome, Vanilla JS canvas, per-user colour, cluster state panel (SSE), Chaos button, log-compensation undo.

Component	Owner	Language	Key Deliverable
RAFT Core Engine	Shrish	Go	Correct state machine + election
Log Replication + WAL	Saffiya	Go	WAL + prevLogIndex + sync-log
Gateway + Docker + Chaos	Rushad	Go + YAML	Zero-downtime failover + chaos
Frontend + Dashboard + Undo	Ayesha	React + JS	Live election visualizer + undo

4. Technology Stack

Layer	Technology	Version	Rationale
Replica + Gateway language	Go	1.22+	Native goroutines for concurrent RAFT timers; air for hot-reload; idiomatic gRPC support
Inter-replica RPC	gRPC + protobuf	latest	Strongly typed, faster than HTTP/JSON, generated client/server stubs eliminate bugs
Gateway transport (clients)	WebSocket (gorilla/websocket)	1.5.x	Low-latency bidirectional for drawing strokes; widely supported
Cluster status feed (frontend)	SSE via Gateway /events/cluster-status	HTTP/1.1	One-directional status push; no WS overhead; browser-native EventSource
Log persistence	Write-ahead log (WAL) on disk	custom	Committed entries survive container restart; replayed on boot before joining cluster
Frontend framework	React 18 + Vite	18.x / 5.x	Dashboard panel, state management, component-based UI chrome

Layer	Technology	Version	Rationale
Canvas rendering	Vanilla JS (HTML5 Canvas API)	native	Direct canvas calls are faster than React-managed DOM for high-freq drawing events
Containerisation	Docker + docker-compose v2	latest	Multi-container orchestration; bind mounts for hot-reload
Hot-reload (replicas)	air	latest	Go-native file watcher; graceful restart on source change
Logging	uber-go/zap	1.27.x	Structured JSON to stdout; fast; industry standard
Metrics	prometheus/client_golang	1.19.x	Native /metrics endpoint; scraped by Prometheus or read directly
Terminal TUI	rivo/tview or charmbracelet/bubbletea	latest	Optional live cluster state in terminal during demo
Frontend styling	Tailwind CSS (CDN)	3.x	Utility-first; no build step needed via CDN script tag

5. Repository Structure

Monorepo — one Git repository. All 4 team members commit here. Branch strategy: main is always deployable. Each person works on a feature branch named feature/<name>/<description> and opens a PR to merge.

```
mini-raft/
├── replica/                    # Shared Go codebase (Shrish + Saffiya)
│   ├── main.go                 # Entry point; reads env vars
│   ├── go.mod / go.sum
│   ├── .air.toml               # air hot-reload config
│   ├── Dockerfile
│   ├── proto/                  # .proto definitions (shared)
│   │   └── raft.proto
│   ├── raft/                   # RAFT state machine (Shrish)
│   │   ├── node.go             # RaftNode struct + state machine
│   │   ├── election.go         # Timer, candidate logic, vote requests
│   │   ├── heartbeat.go        # Leader heartbeat sender
│   │   └── rpc_server.go       # gRPC server: RequestVote, Heartbeat
│   ├── log/                    # Log + WAL (Saffiya)
│   │   ├── entry.go            # LogEntry struct
│   │   ├── log.go              # In-memory log + commit index
│   │   ├── wal.go              # Write-ahead log to disk
│   │   └── replication.go      # AppendEntries, SyncLog RPC handlers
│   ├── metrics/                # Prometheus metrics
│   │   └── metrics.go
│   └── status/                 # /status HTTP handler
│       └── status.go
├── replica1/                   # Bind-mount target (symlink or copy)
```

```
│   └── -> ../replica/          # docker-compose mounts this folder
├── replica2/
│   └── -> ../replica/
├── replica3/
│   └── -> ../replica/
├── gateway/                    # Gateway service (Rushad)
│   ├── main.go
│   ├── go.mod / go.sum
│   ├── .air.toml
│   ├── Dockerfile
│   ├── ws/                     # WebSocket handler
│   │   └── handler.go
│   ├── leader/                 # Leader tracker + router
│   │   └── tracker.go
│   ├── chaos/                  # Chaos mode endpoint
│   │   └── chaos.go
│   ├── sse/                    # SSE /events/cluster-status
│   │   └── sse.go
│   └── metrics/
│       └── metrics.go
├── frontend/                   # React + Vanilla JS (Ayesha)
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   ├── src/
│   │   ├── main.jsx            # React root
│   │   ├── App.jsx             # Layout: Canvas + Sidebar
│   │   ├── canvas/
│   │   │   ├── Canvas.jsx      # Vanilla JS canvas inside React ref
│   │   │   └── drawing.js      # Pure canvas draw logic
│   │   ├── dashboard/
│   │   │   ├── ClusterPanel.jsx
│   │   │   ├── NodeCard.jsx
│   │   │   └── useClusterSSE.js
│   │   ├── toolbar/
│   │   │   └── Toolbar.jsx     # Colour, width, eraser, undo, redo
│   │   ├── chaos/
│   │   │   └── ChaosButton.jsx
│   │   └── ws/
│   │       └── useDrawingWS.js # WebSocket hook
├── docker-compose.yml          # Full system definition (Rushad)
├── docker-compose.dev.yml      # Dev overrides (bind mounts, ports)
├── logs/                       # Bind-mounted WAL + app logs
│   ├── replica1/
│   ├── replica2/
│   └── replica3/
├── docs/
│   ├── architecture.md
│   └── api-spec.md
└── README.md
```

6. Mini-RAFT Protocol Specification

This section is the authoritative specification for the RAFT protocol as implemented in this project. It is a deliberate simplification of full RAFT (Diego Ongaro's PhD thesis). Deviations from the spec defined here must be documented.

6.1 Node States & Transitions

Every replica is always in exactly one of three states:

State	Who sets it	Entry condition	Exit condition
FOLLOWER	All nodes on boot	Node starts; or receives higher-term message; or votes for another candidate	Election timeout fires without receiving heartbeat
CANDIDATE	Follower whose timer fired	Election timeout fires; increments term; votes for self; sends RequestVote to all peers	Receives majority votes (-> LEADER) or receives heartbeat from valid leader (-> FOLLOWER) or loses election (-> FOLLOWER, retry)
LEADER	Candidate who won	Receives >= 2 votes (majority of 3)	Receives message with term > currentTerm (-> FOLLOWER); crash/restart (-> FOLLOWER)

6.2 Persistent State (survives restart via WAL)

- currentTerm: integer — latest term the node has seen. Monotonically increasing.
- votedFor: string | null — candidateId this node voted for in currentTerm. Null if not voted.
- log[]: []LogEntry — the append-only stroke log, including uncommitted entries.

6.3 Volatile State (reset on restart)

- state: FOLLOWER | CANDIDATE | LEADER
- commitIndex: int — index of highest log entry known to be committed. Reset to highest WAL index on restart.
- lastApplied: int — index of highest log entry applied to state machine.
- nextIndex[]: map[peerId]int — (leader only) for each peer, index of next log entry to send.
- matchIndex[]: map[peerId]int — (leader only) for each peer, highest log entry known to be replicated.

6.4 Election Rules (Shrish owns)

- Election timeout is randomly chosen in range 500–800ms on each reset. Different ranges prevent split votes.

- A follower resets its election timer on: receiving a valid heartbeat from the current leader, receiving an AppendEntries with term >= currentTerm, or granting a vote.
- If the timer fires: the node transitions to CANDIDATE, increments currentTerm, sets votedFor = self, sends RequestVote RPCs concurrently to all peers.
- Vote granted if: (a) requester's term >= receiver's currentTerm, (b) receiver has not voted for anyone else in this term, (c) requester's log is at least as up-to-date as receiver's (lastLogIndex >= receiver's lastLogIndex AND lastLogTerm >= receiver's lastLogTerm).
- A CANDIDATE that receives a majority (>= 2 out of 3) votes immediately becomes LEADER and starts sending heartbeats at 150ms interval.
- Split vote: if no candidate reaches majority before the election timeout fires again, the candidate increments term again and starts a new election. The random timeout naturally resolves splits.
- Term demotion: any node that receives a message with term > currentTerm immediately sets currentTerm = that term, clears votedFor, and transitions to FOLLOWER regardless of its current state.
- Heartbeat interval: 150ms. This must be significantly shorter than the election timeout (500ms) to prevent false elections while a leader is alive.

6.5 Log Replication Rules (Saffiya owns)

- Client stroke arrives at Gateway -> forwarded to Leader.
- Leader appends entry to its local log with {term: currentTerm, index: len(log), data: stroke}.
- Leader sends AppendEntries gRPC to all followers concurrently (goroutine per peer).
- AppendEntries carries: term, leaderId, prevLogIndex, prevLogTerm, entries[], leaderCommit.
- Follower validates: if term < currentTerm -> reject. If log[prevLogIndex].term != prevLogTerm -> reject with current log length (triggers catch-up).
- On successful validation: follower appends new entries, updates commitIndex to min(leaderCommit, len(log)-1), responds with success=true.
- Leader waits for majority ACK (>= 2 responses including self). On majority: marks entry as committed, increments commitIndex, broadcasts committed stroke to Gateway.
- Safety invariant: once an entry is committed, it must never be overwritten. Committed entries are also flushed to WAL.

6.6 Catch-Up Protocol (Saffiya owns)

1. Restarted node boots, reads WAL into memory, starts in FOLLOWER state with log = WAL contents.
2. Leader sends AppendEntries with prevLogIndex = len(leader.log) - 2. If follower's log is shorter, prevLogIndex check fails.
3. Follower responds with success=false and includes currentLogLength in the response.
4. Leader sees failure: sets nextIndex[follower] = follower's currentLogLength and calls /sync-log on the follower, sending all committed entries from that index onward.
5. Follower receives SyncLog entries: appends all to log, updates commitIndex, flushes to WAL, responds with success=true.
6. Leader resumes normal AppendEntries — follower is now in sync.
7. Follower participates in future votes and replication normally.

6.7 Safety Rules (both Shrish and Saffiya enforce)

- Committed entries must never be overwritten or deleted.
- A node may only vote for one candidate per term (votedFor persists across crashes via WAL).
- Higher term always wins — term demotion is immediate and unconditional.
- A leader only commits an entry from its current term (no re-committing old term entries directly).
- Log matching: if two logs agree on index and term for an entry, they agree on all previous entries.

7. Data Models

7.1 LogEntry (Go struct — Saffiya)

```
type LogEntry struct {
    Index     int64       `json:"index"`
    Term      int64       `json:"term"`
    Type      EntryType   `json:"type"`   // STROKE | UNDO_COMPENSATION
    StrokeID  string      `json:"strokeId"` // UUID v4
    UserID    string      `json:"userId"`   // session colour identity
    Data      StrokeData  `json:"data"`
    Timestamp int64       `json:"ts"`       // Unix ms
}

type StrokeData struct {
    Points    []Point  `json:"points"`
    Colour    string   `json:"colour"`  // hex e.g. "#E63946"
    Width     float64  `json:"width"`   // px
    Tool      string   `json:"tool"`    // "pen" | "eraser"
}

type EntryType string
const (
    EntryTypeStroke EntryType = "STROKE"
    EntryTypeUndo   EntryType = "UNDO_COMPENSATION"  // data.StrokeID to hide
)
```

7.2 NodeStatus (Go struct — Shrish/Saffiya, served by /status HTTP)

```
type NodeStatus struct {
    ReplicaID   string  `json:"replicaId"`   // "replica1" | "replica2" | "replica3"
    State       string  `json:"state"`       // "FOLLOWER" | "CANDIDATE" | "LEADER"
    Term        int64   `json:"term"`
    LogLength   int     `json:"logLength"`
    CommitIndex int64   `json:"commitIndex"`
    LeaderID    string  `json:"leaderId"`    // empty if unknown
    LastHBMs    int64   `json:"lastHeartbeatMs"` // unix ms of last HB received
    Healthy     bool    `json:"healthy"`
}
```

7.3 WebSocket Message Types (Gateway <-> Frontend)

Type	Direction	Payload fields	Description
STROKE_DRAW	Client -> Gateway	strokeId, userId, points[], colour, width, tool	Client sends a new stroke to be replicated
STROKE_COMMITTED	Gateway -> All clients	strokeId, userId, index, points[], colour, width, tool	Leader committed; broadcast to all
STROKE_UNDO	Client -> Gateway	userId, targetStrokeId	Client requests undo of their own stroke
USER_COLOR_ASSIGNED	Gateway -> Client (unicast)	userId, colour	On WS connect; assigns this session a unique colour
CANVAS_SYNC	Gateway -> Client (unicast)	entries[]	On WS connect; sends full committed log to new client
LEADER_CHANGED	Gateway -> All clients	newLeaderId, term	Notifies clients of leader change (for UI indicator)
ERROR	Gateway -> Client	code, message	General error, e.g. no leader available

7.4 SSE Event Types (Gateway -> Frontend, /events/cluster-status)

Event name	Payload	Frequency
cluster_status	{ replicas: NodeStatus[] }	Every 500ms (gateway polls all replicas)
election_started	{ candidateId, term }	On detection (replica pushes to gateway)
leader_elected	{ leaderId, term }	On detection

8. gRPC Proto Specification (raft.proto)

This is the authoritative proto definition. All four team members must agree this file is frozen by end of Week 1 Day 3. No changes without team discussion.

```
syntax = "proto3";
package raft;
option go_package = "./proto;raft";

// ─── RequestVote ──────────────────────────────────────────────────────
message VoteRequest {
```

```
    int64  term           = 1;  // candidate's term
    string candidate_id   = 2;  // e.g. "replica1"
    int64  last_log_index = 3;  // index of candidate's last log entry
    int64  last_log_term  = 4;  // term of candidate's last log entry
}

message VoteResponse {
    int64 term          = 1;  // receiver's currentTerm (for candidate to update)
    bool  vote_granted  = 2;
}

// ─── AppendEntries ──────────────────────────────────────────────────────
message AppendEntriesRequest {
    int64         term           = 1;
    string        leader_id      = 2;
    int64         prev_log_index = 3;
    int64         prev_log_term  = 4;
    repeated      LogEntry entries = 5;
    int64         leader_commit  = 6;
}

message AppendEntriesResponse {
    int64 term            = 1;
    bool  success         = 2;
    int64 conflict_index  = 3;  // follower's log length if rejected
}

// ─── Heartbeat ──────────────────────────────────────────────────────────
message HeartbeatRequest {
    int64  term       = 1;
    string leader_id  = 2;
    int64  commit_index = 3;
}

message HeartbeatResponse {
    int64 term    = 1;
    bool  success = 2;
}

// ─── SyncLog ────────────────────────────────────────────────────────────
message SyncLogRequest {
    int64  from_index = 1;  // follower requests entries from this index
    string replica_id = 2;
}

message SyncLogResponse {
    repeated LogEntry entries     = 1;
    int64             commit_index = 2;
}
```

```
// ─── LogEntry ──────────────────────────────────────────────────────────
message LogEntry {
    int64  index      = 1;
    int64  term       = 2;
    string type       = 3;  // "STROKE" | "UNDO_COMPENSATION"
    string stroke_id  = 4;  // UUID v4
    string user_id    = 5;
    bytes  data       = 6;  // JSON-encoded StrokeData
    int64  timestamp  = 7;  // Unix ms
}

// ─── RPC Service ───────────────────────────────────────────────────────
service RaftService {
    rpc RequestVote   (VoteRequest)          returns (VoteResponse);
    rpc AppendEntries (AppendEntriesRequest) returns (AppendEntriesResponse);
    rpc Heartbeat     (HeartbeatRequest)     returns (HeartbeatResponse);
    rpc SyncLog       (SyncLogRequest)       returns (SyncLogResponse);
}
```

9. HTTP REST API Specification

9.1 Replica HTTP Endpoints (all 3 replicas expose these)

Method	Path	Owner	Description	Response
GET	/status	Shrish	Returns NodeStatus JSON. Polled by Gateway every 500ms.	200 NodeStatus JSON
GET	/metrics	Saffiya	Prometheus metrics endpoint. Scraped by Prometheus or curl.	200 text/plain Prometheus format
GET	/health	Shrish	Simple liveness check for docker-compose healthcheck.	200 {ok:true}
POST	/stroke	Saffiya	Gateway forwards client stroke to leader replica. Body: LogEntry (without index/term). Only accepted if node is LEADER.	202 Accepted | 503 Not Leader

9.2 Gateway HTTP Endpoints

Method	Path	Owner	Description	Response
GET	/ws	Rushad	WebSocket upgrade endpoint. Browser connects here.	101 Switching Protocols
GET	/events/cluster-status	Rushad	SSE stream. Frontend subscribes via EventSource. Pushes cluster_status every 500ms.	200 text/event-stream

Method	Path	Owner	Description	Response
POST	/chaos	Rushad	Triggers chaos. Body: {target: 'random'|'replica1'|'replica2'|'replica3', mode: 'graceful'|'hard'|'random'}. Requires Docker socket mount.	200 {killed: replicaId, mode}
GET	/leader	Rushad	Returns current known leader ID and term.	200 {leaderId, term}
GET	/metrics	Rushad	Gateway Prometheus metrics.	200 text/plain
GET	/health	Rushad	Gateway liveness.	200 {ok:true}

10. Component Specifications

10.1 RAFT Core Engine — Shrish

Ownership scope

Files: replica/raft/node.go, election.go, heartbeat.go, rpc_server.go, status/status.go. You are responsible for the state machine, all timer logic, vote RPC client/server, heartbeat client/server, and the /status + /health HTTP handlers.

10.1.1 RaftNode Struct

```
type RaftNode struct {
    mu          sync.Mutex
    id          string          // "replica1" etc.
    peers       []string        // gRPC addresses of other replicas
    state       NodeState       // FOLLOWER | CANDIDATE | LEADER
    currentTerm int64
    votedFor    string
    log         *log.RaftLog    // owned by Saffiya; pointer injected
    commitIndex int64
    lastApplied int64
    nextIndex   map[string]int64 // leader only
    matchIndex  map[string]int64 // leader only
    electionTimer  *time.Timer
    heartbeatStop  chan struct{}
    logger         *zap.Logger
    metrics        *metrics.ReplicaMetrics
}
```

10.1.2 Election Timer Logic

This is the most concurrency-sensitive piece. Use the following pattern exactly:

- Store the timer as a *time.Timer field on RaftNode.
- resetElectionTimer() must: (1) call timer.Stop(), (2) drain the channel with select { case <-timer.C: default: }, (3) call timer.Reset(randomTimeout()). This is the standard Go pattern to safely reset a timer.

- randomTimeout() returns a duration in [500ms, 800ms] using rand.Intn(300) + 500.
- Run the election loop in a goroutine: for { select { case <-node.electionTimer.C: node.startElection() } }.
- Acquire node.mu before reading/writing any state. Release before any blocking I/O (gRPC calls).

10.1.3 startElection() Flow

8. Lock mu; transition to CANDIDATE; increment currentTerm; set votedFor = self; snapshot term and peers; release mu.
9. Reset election timer (in case this election also times out).
10. Send RequestVote gRPC concurrently to all peers using goroutines. Use a context with 300ms timeout per RPC.
11. Count votes (including self vote) using an atomic counter or a votes channel.
12. If votes >= 2: lock mu, verify still CANDIDATE and term hasn't changed, transition to LEADER, initialise nextIndex = len(log) for all peers, matchIndex = 0, stop election timer, start heartbeat loop.
13. If receives VoteResponse with term > currentTerm: lock mu, step down to FOLLOWER, update term, clear votedFor.
14. If election timer fires again before majority: start new election (term increments again).

10.1.4 Heartbeat Sender

- Run in a dedicated goroutine started when node becomes LEADER.
- Send HeartbeatRequest to all peers every 150ms via gRPC concurrently.
- On HeartbeatResponse with term > currentTerm: step down to FOLLOWER immediately.
- Stop heartbeat goroutine when node steps down, using heartbeatStop channel.
- Heartbeat also carries leaderCommit so followers can advance their commitIndex.

10.1.5 /status HTTP Handler

Runs on a separate HTTP port (e.g. :8081 alongside gRPC on :9001). Returns NodeStatus JSON. This endpoint is polled by Gateway every 500ms — it must be fast (< 1ms), so only acquire mu briefly and copy the needed fields.

10.1.6 Prometheus Metrics (Shrish exposes)

- raft_term_total — gauge: current term
- raft_state — gauge: 0=follower, 1=candidate, 2=leader
- raft_elections_total — counter: number of elections started
- raft_votes_granted_total — counter
- raft_heartbeats_sent_total — counter (leader only)
- raft_heartbeat_latency_seconds — histogram

10.2 Log Replication + WAL + Catch-Up — Saffiya

Ownership scope

Files: replica/log/entry.go, log.go, wal.go, replication.go, metrics/metrics.go (log metrics). You own the append-only log data structure, WAL persistence, AppendEntries gRPC handler, /sync-log gRPC handler, and commit logic.

10.2.1 RaftLog Struct

```
type RaftLog struct {
    mu          sync.RWMutex
    entries     []LogEntry
    commitIndex int64
    wal         *WAL
    logger      *zap.Logger
}
```

- AppendEntry(entry LogEntry) error — appends to in-memory slice and WAL.
- GetEntry(index int64) (LogEntry, bool)
- GetEntriesFrom(fromIndex int64) []LogEntry — used by SyncLog.
- Commit(index int64) error — marks entry as committed; flushes WAL.
- LastIndex() int64, LastTerm() int64
- LoadFromWAL() error — called on boot to replay WAL into memory.

10.2.2 Write-Ahead Log (WAL)

The WAL is a simple append-only file at /data/wal.log (bind-mounted to logs/<replicaN>/ on host). Each line is a JSON-encoded WAL record. On boot, the replica replays the WAL to restore log state before joining the cluster.

```
type WALRecord struct {
    Type    string   `json:"type"`   // "TERM" | "VOTE" | "ENTRY" | "COMMIT"
    Term    int64    `json:"term,omitempty"`
    VotedFor string  `json:"votedFor,omitempty"`
    Entry   *LogEntry `json:"entry,omitempty"`
    CommitIndex int64 `json:"commitIndex,omitempty"`
}
```

- WAL record types: TERM (persist currentTerm), VOTE (persist votedFor), ENTRY (persist new log entry), COMMIT (persist commitIndex advance).
- WAL file opened with O_APPEND | O_CREATE | O_WRONLY. Each write is followed by Sync() to guarantee durability.
- On startup: replay all TERM records (take highest), all VOTE records (take latest), all ENTRY records in order, all COMMIT records (take highest). This fully restores persistent state.
- votedFor and currentTerm are also stored in WAL so they survive restarts — this is a RAFT safety requirement.

10.2.3 AppendEntries gRPC Handler

15. Acquire log.mu write lock.
16. If request.term < currentTerm: respond success=false, term=currentTerm.
17. Reset election timer (valid message from leader).
18. If prevLogIndex > 0 and log[prevLogIndex].term != prevLogTerm: respond success=false, conflictIndex=len(log).
19. Truncate log at conflictIndex if entries conflict (same index, different term).
20. Append new entries to log and WAL.
21. If leaderCommit > commitIndex: set commitIndex = min(leaderCommit, len(log)-1), flush COMMIT to WAL.
22. Respond success=true.

10.2.4 SyncLog gRPC Handler

Called by leader on a follower that is behind. Returns all committed entries from fromIndex onward. The follower appends these, updates commitIndex, flushes WAL. This is the catch-up protocol.

10.2.5 Leader Commit Logic

After leader sends AppendEntries to followers, it waits for majority ACK using a result channel. Pattern:

- Leader sends AppendEntries to N-1 peers concurrently.
- Uses a buffered channel of size N-1 for responses.
- Counts ACKs as they arrive. As soon as acks+1 (self) >= majority: commit.
- Uses a 500ms overall timeout — if majority not reached, entry stays uncommitted.
- On commit: update commitIndex, flush WAL, notify Gateway via HTTP POST /committed.

10.2.6 Prometheus Metrics (Saffiya exposes)

- raft_log_length — gauge
- raft_commit_index — gauge
- raft_entries_appended_total — counter
- raft_entries_committed_total — counter
- raft_sync_log_requests_total — counter
- raft_wal_writes_total — counter
- raft_wal_bytes_written_total — counter

10.3 Gateway + Docker Infrastructure + Chaos Mode — Rushad

Ownership scope

Files: gateway/ (entire service), docker-compose.yml, docker-compose.dev.yml, replica/.air.toml, all Dockerfiles. You own the WebSocket server, leader tracker, failover handling, SSE feed, Chaos endpoint, and the entire Docker infrastructure.

10.3.1 Leader Tracker

The Gateway maintains a LeaderTracker that continuously knows who the current leader is. This is critical for routing strokes correctly.

```
type LeaderTracker struct {
    mu       sync.RWMutex
    leaderID string
    term     int64
    replicas []ReplicaConfig  // {id, statusURL, strokeURL}
    logger   *zap.Logger
}
```

- Background goroutine polls /status on all 3 replicas every 500ms concurrently.
- Whichever replica responds with state=LEADER and highest term is the current leader.
- If no replica is LEADER (election in progress): Gateway buffers incoming strokes for up to 2 seconds, retrying every 100ms. If still no leader after 2s: respond to client with ERROR message.
- Thread-safe: all reads/writes to leaderID and term go through RWMutex.
- On leader change: pushes LEADER_CHANGED WebSocket message to all connected clients.

10.3.2 WebSocket Handler

- On connect: assign userId (UUID v4) and colour (from a deterministic palette of 16 distinct colours, cycling). Send USER_COLOR_ASSIGNED unicast message.
- Send CANVAS_SYNC unicast: HTTP GET /entries on current leader to get all committed entries; send as CANVAS_SYNC WebSocket message.
- On STROKE_DRAW message: extract stroke data, assign strokeId (UUID v4), forward to leader via POST /stroke. On 503 (not leader): re-query leader and retry once.
- On STROKE_UNDO message: validate userId == entry's userId (can only undo own strokes). Create UNDO_COMPENSATION LogEntry. Forward to leader.
- On leader committing an entry: receives POST /committed callback from replica. Broadcasts STROKE_COMMITTED to all connected clients.
- Client connections are stored in a sync.Map[userId]WebSocketConn. Thread-safe broadcast iterates all connections.
- Ping/pong keepalive every 30s to detect stale connections.

10.3.3 SSE Feed (/events/cluster-status)

- Browser connects with EventSource('/events/cluster-status').
- Gateway polls all 3 replica /status endpoints every 500ms.
- Aggregates into ClusterStatus{replicas: [NodeStatus, NodeStatus, NodeStatus]}.
- Pushes as SSE event: data: {JSON} followed by double newline.
- Also pushes election_started and leader_elected events when leader changes detected.
- Connections registered in a sync.Map; garbage collected on client disconnect.

10.3.4 Chaos Mode

- Gateway mounts /var/run/docker.sock as a volume (defined in docker-compose.yml).

- Uses Docker SDK for Go (docker/docker/client) to issue stop or kill commands.
- POST /chaos body: { target: 'random'|'replica1'|'replica2'|'replica3', mode: 'graceful'|'hard'|'random' }.
- If target=random: picks uniformly from healthy replicas (avoids killing already-dead ones).
- If mode=graceful: calls ContainerStop(ctx, containerName, &timeout) where timeout=5s.
- If mode=hard: calls ContainerKill(ctx, containerName, 'SIGKILL').
- If mode=random: randomly picks graceful or hard with 50/50 probability.
- Returns {killed: replicaId, mode: 'graceful'|'hard', timestamp: ...}.
- Chaos mode is intentionally unauthenticated — this is a demo feature.

10.3.5 docker-compose.yml Specification

Full definition of the entire system. Must be committable and runnable with a single docker-compose up --build.

```
services:
  gateway:
    build: ./gateway
    ports: ["8080:8080"]
    environment:
      - REPLICA_ADDRS=replica1:9001,replica2:9001,replica3:9001
      - REPLICA_HTTP=http://replica1:8081,http://replica2:8081,http://replica3:8081
      - LOG_LEVEL=info
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    depends_on:
      replica1: { condition: service_healthy }
      replica2: { condition: service_healthy }
      replica3: { condition: service_healthy }
    networks: [raft-net]

  replica1:
    build: ./replica
    environment:
      - REPLICA_ID=replica1
      - PEERS=replica2:9001,replica3:9001
      - GRPC_PORT=9001
      - HTTP_PORT=8081
      - DATA_DIR=/data
      - LOG_LEVEL=info
    volumes:
      - ./replica1:/app/src          # bind-mount for air hot-reload
      - ./logs/replica1:/data         # bind-mount for WAL + logs
    ports: ["9001:9001", "8081:8081"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8081/health"]
      interval: 5s
      timeout: 3s
      retries: 5
```

```
    networks: [raft-net]
  # replica2 and replica3: identical, change REPLICA_ID and PEERS

  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    networks: [raft-net]

networks:
  raft-net:
    driver: bridge
```

10.3.6 air Hot-Reload Config (.air.toml)

```
[build]
  cmd = "go build -o ./tmp/main ."
  bin = "./tmp/main"
  include_ext = ["go", "proto"]
  exclude_dir = ["tmp", "vendor"]
  delay = 1000  # ms — debounce file changes
[misc]
  clean_on_exit = true
```

10.3.7 Gateway Prometheus Metrics (Rushad exposes)

- gateway_websocket_connections_total — gauge: current active WS connections
- gateway_strokes_forwarded_total — counter
- gateway_leader_changes_total — counter
- gateway_failover_duration_seconds — histogram: time from leader death to new leader detected
- gateway_chaos_events_total — counter, labelled by mode

10.4 Frontend + Live Dashboard + Undo/Redo — Ayesha

Ownership scope

Files: frontend/ (entire directory). You own the React app scaffold, Vanilla JS canvas rendering, WebSocket client hook, SSE cluster status hook, toolbar, cluster panel, chaos button, undo/redo stack, and all styling.

10.4.1 App Layout

The app is a single page with two panels: the drawing canvas (takes 75% of viewport width) and the cluster dashboard sidebar (25%). The toolbar sits above the canvas.

```
App.jsx
  ├── <Toolbar />              colour picker, width slider, eraser, undo, redo
  ├── <Canvas />               Vanilla JS canvas inside a React ref
  |     └── useDrawingWS.js    WebSocket hook
```

```
└── <ClusterPanel />         SSE-fed sidebar
      ├── <NodeCard />        one per replica
      ├── <ChaosButton />
      └── <ConnectionStatus /> shows current leader
```

10.4.2 Canvas (Vanilla JS)

- Canvas element is managed via a React ref (useRef). React never re-renders the canvas — all drawing is done via direct canvas API calls in drawing.js.
- Drawing state: Map<strokeId, StrokeData> for all committed strokes. On each animation frame, if dirty, redraw all strokes from scratch (simple but correct; optimise later if slow).
- Mouse/touch event handlers: onmousedown starts a stroke, onmousemove appends points, onmouseup finalises and sends via WebSocket.
- Optimistic rendering: the user's own stroke is drawn immediately on screen (before RAFT commits) using a pending layer. When STROKE_COMMITTED arrives, the pending layer clears and the committed stroke renders in the main layer. This eliminates perceived lag.
- On STROKE_COMMITTED (remote): parse stroke, add to committed map, set dirty=true, next animation frame redraws.
- On UNDO_COMPENSATION committed: remove the targetStrokeId from committed map, set dirty=true.
- Eraser: sets tool='eraser' and colour='#FFFFFF' (or background colour); the stroke is a normal RAFT entry.

10.4.3 Per-User Colour Identity

- On WS connect, Gateway sends USER_COLOR_ASSIGNED: {userId, colour}.
- Frontend stores userId and colour in React state. All strokes sent from this tab use this colour.
- Colour is shown in the toolbar as the active colour (user can't change it — it's their identity).
- Incoming strokes from other users render in their assigned colour (included in STROKE_COMMITTED payload).
- Palette: 16 distinct, accessible, non-clashing colours. Gateway cycles through them as clients connect.

10.4.4 Toolbar

- Stroke width: a range slider (1–20px).
- Eraser: toggle button. When active, tool='eraser'.
- Undo (Ctrl+Z or button): pops the top of the local undo stack (list of strokeIds drawn by this user in this session), sends STROKE_UNDO via WebSocket.
- Redo (Ctrl+Y or button): re-sends the most recently undone stroke as a new STROKE_DRAW. Re-adds to undo stack.
- User colour badge: non-editable circle showing this session's assigned colour.

10.4.5 Undo/Redo via Log Compensation

This is the technically interesting bonus. Undo does NOT delete a log entry — it appends a new UNDO_COMPENSATION entry to the RAFT log. This preserves append-only invariant.

- Client-side: undoStack = [] (array of {strokeId, data}). Every time this user's stroke is committed, push to undoStack.
- On undo: pop from undoStack, push to redoStack, send STROKE_UNDO{userId, targetStrokeId} to Gateway.
- Gateway creates LogEntry{type: UNDO_COMPENSATION, strokeId: newUUID, userId, data: {targetStrokeId}}.
- This goes through full RAFT pipeline — AppendEntries, majority ACK, commit.
- On STROKE_COMMITTED with type=UNDO_COMPENSATION: all clients remove targetStrokeId from their committed map and redraw.
- On redo: pop from redoStack, re-send original stroke data as new STROKE_DRAW with new strokeId.
- Undo stack is client-side only — scoped to this session. Refresh = lost undo history (acceptable).

10.4.6 Live Cluster Dashboard Panel

This is the demo's showstopper. An embedded sidebar that shows the cluster state updating in real time during elections.

- useClusterSSE.js: connects to GET /events/cluster-status via the browser's native EventSource API. Parses incoming cluster_status events. Updates React state: replicas: NodeStatus[].
- ClusterPanel.jsx: renders three NodeCard components, one per replica, plus a ConnectionStatus bar.
- NodeCard.jsx: displays replica ID, state (FOLLOWER / CANDIDATE / LEADER with coloured badge), term number, log length, commit index, and a 'last heartbeat' timer. State CANDIDATE pulses with a CSS animation. State LEADER glows green. State FOLLOWER is grey.
- When election_started SSE event arrives: NodeCard for that replica shows CANDIDATE state immediately with pulsing animation.
- When leader_elected SSE event arrives: the new leader's card glows, others turn grey.
- The term number incrementing in real time during an election is visually dramatic.
- Log length counter ticks up with every committed stroke — shows the append-only log growing.

10.4.7 Chaos Mode Button

- Large, prominently styled button in the cluster panel: 'Unleash Chaos'.
- Target selector: dropdown — Random Replica / Replica 1 / Replica 2 / Replica 3.
- Kill mode toggle: Graceful / Hard / Surprise (random).
- On click: POST /chaos to Gateway. Shows a toast notification: 'Killed replica2 (hard kill)'.
- The cluster panel immediately shows the killed replica as unreachable (no /status response).
- An election starts (visible via pulsing CANDIDATE badge) and a new leader emerges.
- Drawing continues without interruption if the canvas is being drawn during chaos.

10.4.8 Connection Status Indicator

- Banner at the top of the canvas: shows 'Connected — Leader: replica1 (Term 3)'.
- During election: 'Election in progress... (Term 4)' with a spinner.

- If Gateway WS disconnects: 'Reconnecting...' with exponential backoff.
- Colour-coded: green=healthy, amber=election, red=disconnected.

11. Observability Stack

11.1 Structured Logging (zap)

All services log structured JSON to stdout. Docker captures this via docker logs <container>. Every log line includes: timestamp, level, service, replicaId, term, state, message, and relevant fields.

```
// Example zap log line (JSON):
{"ts":"2024-01-15T10:23:45Z","level":"info","service":"replica",
 "replicaId":"replica1","term":4,"state":"LEADER",
 "msg":"entry committed","index":42,"strokeId":"abc-123"}
```

- Election events: WARN level — 'election started', 'vote granted', 'vote rejected', 'became leader', 'stepped down'.
- Replication events: INFO level — 'AppendEntries sent', 'entry committed', 'sync-log completed'.
- WAL events: DEBUG level — 'WAL write', 'WAL flush'.
- Gateway events: INFO level — 'client connected', 'leader changed', 'chaos triggered', 'stroke forwarded'.

11.2 Log Files on Bind-Mounted Volumes

In addition to stdout, each replica writes its zap logs to /data/app.log (which is bind-mounted to logs/replica1/app.log on the host). The WAL is at /data/wal.log. This means you can tail -f logs/replica1/app.log from the host during the demo to show real-time replica activity.

11.3 Prometheus Metrics

Every replica exposes GET /metrics in Prometheus text format on HTTP port 8081. The gateway exposes the same on port 8080. During the demo, you can curl http://localhost:8081/metrics to show live metrics. Optionally, run a Prometheus + Grafana stack alongside via docker-compose.

11.4 Terminal TUI (Optional — Bonus)

A standalone Go binary in tools/tui/ that polls all /status endpoints every 500ms and renders a terminal dashboard using tview or bubbletea. Shows: a table with replica states, term, log length, commit index; a scrollable log tail; and a command input for triggering chaos. Run alongside docker-compose in a separate terminal during the demo for a spectacular effect.

12. Interface Contracts (Lock Down by Week 1 Day 3)

These are the exact boundaries between components. All four team members must agree on these before writing any implementation code. Changing these after Week 1 breaks everyone's work.

Interface	Between	What must be frozen	Owner of definition
gRPC proto	Shrish <-> Saffiya	All 4 RPC method signatures and message fields in raft.proto	Both — joint agreement
NodeStatus JSON	Shrish -> Rushad	Exact JSON field names and types returned by /status	Shrish defines, Rushad consumes
/stroke HTTP	Rushad -> Saffiya	POST body format, 202/503 response codes	Saffiya defines, Rushad calls
/committed callback	Saffiya -> Rushad	How replica notifies Gateway of committed entries	Saffiya defines, Rushad handles
WS message types	Rushad <-> Ayesha	All message type strings and payload shapes (section 7.3)	Rushad defines, Ayesha consumes
SSE event format	Rushad -> Ayesha	Event names and JSON payload shapes (section 7.4)	Rushad defines, Ayesha consumes
/chaos endpoint	Ayesha -> Rushad	POST body and response shape (section 9.2)	Rushad defines, Ayesha calls
/events/cluster-status	Rushad -> Ayesha	SSE stream URL, event names, payload	Rushad defines, Ayesha subscribes

Critical note on /committed callback

When the leader commits an entry, it must notify the Gateway so the Gateway can broadcast STROKE_COMMITTED to all WebSocket clients. Two options: (A) Leader POSTs to Gateway's /committed endpoint. (B) Gateway polls each replica's /committed-entries endpoint. Option A is faster and recommended. Rushad must expose POST /internal/committed on the Gateway, and Saffiya must call it after a commit. This URL is internal only — not exposed to browsers.

13. Week-by-Week Execution Roadmap

Week 1 — Design, Contracts, Scaffolding (Days 1–7)

Goal

By end of Week 1: every person has a running skeleton of their component, the proto file is frozen, all interfaces are agreed, and docker-compose up brings up 4 healthy containers (even if they do nothing yet).

Days 1–2: Team kickoff + Proto freeze

- All 4 members: read this PRD completely. No exceptions.
- Shrish + Saffiya: finalise raft.proto together. Commit to repo. Generate Go stubs.
- Rushad: set up monorepo structure, initial docker-compose.yml, Dockerfiles for all services, .air.toml configs. Verify air hot-reload works.

- Ayesha: scaffold React + Vite frontend. Verify npm run dev works. Create App.jsx layout skeleton with empty Canvas and ClusterPanel.
- Agree on NodeStatus JSON (Shrish defines, shares with Rushad and Ayesha).
- Agree on WebSocket message types (Rushad shares with Ayesha).

Days 3–4: Skeleton implementation

- Shrish: implement RaftNode struct, state enum, election timer goroutine (no gRPC yet — just logs when timer fires). Implement /status and /health HTTP handlers returning hardcoded data.
- Saffiya: implement RaftLog struct, WAL file write/read/replay. Write unit tests for WAL replay. Implement AppendEntries gRPC handler (stub — just log and return success for now).
- Rushad: implement Gateway WebSocket server (accept connections, echo messages). Implement LeaderTracker polling loop (polls /status — returns dummy data for now). Wire up docker-compose so all 4 containers start healthy.
- Ayesha: implement useDrawingWS hook (connects to WS, sends/receives raw messages). Implement basic canvas drawing (local only, no networking). Set up SSE EventSource connection to /events/cluster-status.

Days 5–7: Integration test + architecture document

- Team: run docker-compose up. Verify all 4 containers start and pass healthchecks. Verify Gateway can reach replica /status endpoints. Verify frontend loads and canvas draws locally.
- All 4: write architecture document (2–3 pages): cluster diagram, state transitions, API definition, failure handling. Divide sections by component.
- Shrish: write failure scenarios list (what happens if replica1 dies during election? what if leader dies mid-commit? what if 2 replicas die simultaneously?).

Week 2 — Core Implementation (Days 8–14)

Goal

By end of Week 2: elections work end-to-end, strokes replicate correctly, the drawing board shows strokes from multiple tabs, and WAL replay works after a restart.

Days 8–9: Election + Heartbeat

- Shrish: implement full election logic — startElection(), RequestVote gRPC client/server, vote counting, leader transition. Implement heartbeat sender and HeartbeatRequest gRPC handler.
- Saffiya: implement WAL persistence for currentTerm and votedFor (TERM and VOTE records). Wire into Shrish's election code.
- Test: docker-compose up; observe logs; verify a leader emerges; kill leader container; verify new leader elected within 1 second.

Days 10–11: Log Replication + Gateway routing

- Saffiya: implement full AppendEntries gRPC client (leader sends to followers), majority ACK counting, commit logic, WAL ENTRY + COMMIT records, /sync-log handler.

- Rushad: implement full stroke routing in Gateway — receive STROKE_DRAW from WS client, POST to current leader's /stroke endpoint, receive /internal/committed callback, broadcast STROKE_COMMITTED to all WS clients.
- Test: open 2 browser tabs; draw in one; verify stroke appears in both. Kill leader; verify drawing resumes after failover.

Days 12–13: Catch-up + WAL replay

- Saffiya: implement full catch-up protocol — follower detects prevLogIndex mismatch, reports log length, leader calls /sync-log, follower appends and re-syncs.
- Shrish: implement WAL replay on boot — replica1 starts, reads WAL, restores log, rejoins cluster.
- Test: draw 20 strokes. Kill replica1. Draw 20 more. Restart replica1. Verify replica1 catches up to 40 strokes via sync-log. Verify all clients see consistent state.

Day 14: Frontend canvas integration + per-user colour

- Ayesha: implement USER_COLOR_ASSIGNED handling, CANVAS_SYNC on connect (full log replay), STROKE_COMMITTED rendering, optimistic local rendering.
- Test: 3 browser tabs; each gets different colour; strokes from all 3 appear correctly on all screens.

Week 3 — Reliability, Polish, Bonus, Demo (Days 15–21)

Goal

By end of Week 3: zero-downtime hot-reload works, chaos mode is functional, dashboard is live, undo/redo works, demo video is recorded.

Days 15–16: Chaos mode + dashboard

- Rushad: implement POST /chaos endpoint with Docker SDK. Test graceful and hard kills. Verify Gateway re-routes correctly.
- Ayesha: implement ClusterPanel with NodeCard components, useClusterSSE hook, state badges, election animation. Implement ChaosButton with target + mode selectors.
- Test: open drawing board; click Unleash Chaos; watch election happen in dashboard while drawing continues.

Days 17–18: Undo/Redo + hot-reload

- Ayesha: implement client-side undo/redo stack, STROKE_UNDO message sending, UNDO_COMPENSATION rendering on commit.
- Shrish: verify hot-reload triggers election correctly. Test: edit replica/raft/heartbeat.go; save; watch container restart; watch election in logs; verify canvas unaffected.
- Test undo: draw 5 strokes; undo 3; verify 3 are gone from all clients; redo 2; verify 2 return.

Days 19–20: Stress testing + edge cases

- All: run chaotic test — 3 clients drawing simultaneously; kill leader; kill another replica (only 1 remains — system should stall, not corrupt); restart both; verify catch-up; verify all clients in sync.
- Verify: split vote scenario (if possible to reproduce with 3 nodes — adjust timeouts temporarily).
- Verify: draw during election — strokes drawn while no leader must be buffered and applied after election completes.
- Verify: WAL — kill all 3 replicas; restart; verify all committed strokes visible on reconnect.

Day 21: Demo video recording

- Record 8–10 minute video following the required checklist: multi-tab drawing, kill leader, failover, hot-reload, consistent state, chaotic conditions.
- Show the cluster dashboard throughout — it makes elections visible and dramatic.
- Demonstrate undo/redo working across clients.
- Use Chaos Mode button for live failure injection — do not manually docker stop in the video.
- Each person narrates their own component section of the video.

14. Risk Assessment & Mitigation

Risk	Likelihood	Impact	Owner	Mitigation
gRPC setup takes longer than expected for Go beginners	High	High	Shrish + Saffiya	Allocate Day 1 entirely to protoc codegen and a working hello-world gRPC call between two replicas. Don't proceed until this works.
Election timer reset bug causes false elections or no elections	High	High	Shrish	Use the exact timer.Stop() + drain + Reset() pattern documented in section 10.1.2. Write a unit test for the timer.
Gateway loses stroke during failover	Medium	High	Rushad	Buffer incoming strokes in a channel with 2s retry window while polling for new leader. Return ERROR to client only after 2s timeout.
WAL replay order incorrect causing log corruption	Medium	High	Saffiya	Unit test WAL: write 50 entries, close, reopen, replay, verify exact same entries in exact same order.
Split brain: two leaders simultaneously think they are leader	Low	Critical	Shrish	Term demotion is the defence — any node receiving a message with higher term immediately steps down. Write a test that sends a higher-term message to a leader and verifies stepdown.
Docker socket permission denied for Chaos Mode	Medium	Low	Rushad	Add user: root or privileged: true to gateway service in docker-compose. Or chmod 666 /var/run/docker.sock.
Frontend canvas flickers during failover	Medium	Medium	Ayesha	Optimistic rendering layer + committed layer separation. Never clear committed layer;

Risk	Likelihood	Impact	Owner	Mitigation
				only update it when STROKE_COMMITTED arrives.
air hot-reload triggers before file write is complete causing build error	Low	Low	Rushad	Set delay = 1000ms in .air.toml to debounce rapid file changes.
3 weeks not enough for TUI bonus	Medium	Low	All	TUI is last priority. Cut it if Week 3 is tight. Dashboard panel is higher value.

15. Viva Preparation — Per Person

Viva is individual. Each person will be asked deep questions about their component AND basic questions about the full system. Prepare both.

15.1 Shrish — RAFT Core Engine

Must be able to answer without hesitation:

- Walk me through exactly what happens from the moment an election timeout fires to the moment a new leader is elected.
- What is a split vote? How do you handle it? Why does randomising timeouts help?
- Why can't a node vote for two candidates in the same term?
- What does 'higher term always wins' mean? Show me in your code where you implement it.
- What is the heartbeat interval and why must it be shorter than the election timeout?
- What happens if the leader receives a message with a term higher than its own?
- Why do you use goroutines for election and heartbeat? How do you prevent data races?
- What is the purpose of the mu sync.Mutex? What would happen without it?

15.2 Saffiya — Log Replication + WAL + Catch-Up

Must be able to answer without hesitation:

- Explain prevLogIndex and prevLogTerm. Why does the follower check these? What happens if they don't match?
- Walk me through the catch-up protocol step by step for a node that missed 20 entries.
- What is a write-ahead log? Why do we need it? What would happen without WAL?
- What does 'committed' mean in RAFT? Why can a committed entry never be overwritten?
- How does the leader know when to commit an entry? What is the majority quorum?
- What is a UNDO_COMPENSATION entry? How does it preserve the append-only invariant?
- If the leader commits an entry and then crashes before telling followers, what happens?
- Show me the WAL replay logic. How do you restore state after a restart?

15.3 Rushad — Gateway + Docker + Chaos

Must be able to answer without hesitation:

- How does the Gateway know who the current leader is? What happens during an election?
- What happens to a stroke that arrives at the Gateway while an election is in progress?
- Explain bind mounts. How do they enable hot-reload? What is air and how does it work?
- How does docker-compose service discovery work? How does replica1 find replica2?
- What is the Docker socket and why does Chaos Mode need it mounted?
- What is the difference between docker stop and docker kill? Which sends SIGTERM vs SIGKILL?
- What is a Docker bridge network? Why do you need it here?
- What are healthchecks in docker-compose and why do you have depends_on with conditions?

15.4 Ayesha — Frontend + Dashboard + Undo/Redo

Must be able to answer without hesitation:

- Why do you use Vanilla JS for the canvas instead of React? What is the performance difference?
- What is optimistic rendering? How does your canvas avoid lag for the local user?
- What is SSE (Server-Sent Events)? How does it differ from WebSocket? Why did you choose SSE for the dashboard?
- Explain log compensation for undo. Why don't you just delete the entry?
- How does per-user colour identity work? What is the flow from WS connect to first stroke?
- What happens to the canvas if the Gateway WebSocket disconnects? How do you reconnect?
- How does CANVAS_SYNC work? Why do you need it when a new client connects?
- What is EventSource in the browser? Show me how useClusterSSE.js uses it.

15.5 Questions all 4 must answer

- What is RAFT? What problem does it solve?
- What are the three node states in RAFT and when does a node enter each?
- What is the CAP theorem? Where does your system sit on the spectrum?
- What is the difference between eventual consistency and strong consistency?
- How is your system similar to etcd in Kubernetes?
- What would happen if 2 out of 3 replicas died simultaneously?
- What is zero-downtime deployment? How does your hot-reload simulate it?
- What is a quorum? Why is majority (n/2 + 1) the threshold?

16. Demo Video Checklist (8–10 minutes)

Follow this exact sequence for the video. Each section is narrated by the component owner.

Time	Section	Narrator	What to show
0:00–1:00	System intro	Shrish	docker-compose up; all 4 containers healthy; cluster dashboard showing leader (green) and 2 followers (grey)
1:00–2:30	Multi-user drawing	Ayesha	3 browser tabs open; each gets different colour; draw in all 3 simultaneously; show strokes appearing on all screens in real time
2:30–3:30	Undo/Redo demo	Ayesha	Draw 5 strokes; undo 3 one by one (show disappearing from all tabs); redo 2 (show reappearing)
3:30–5:00	Chaos Mode — kill leader	Rushad	Click Unleash Chaos targeting current leader; show dashboard: leader goes grey, another pulses CANDIDATE, new leader elected; drawing never stops
5:00–6:00	Hot-reload demo	Shrish	Edit replica/raft/heartbeat.go (add a comment); save; container restarts; dashboard shows election; drawing continues uninterrupted
6:00–7:00	WAL + catch-up demo	Saffiya	Kill one replica; draw 10 more strokes; restart replica; show sync-log in logs; show dashboard: replica rejoins; verify canvas consistent
7:00–8:30	Chaos stress test	All	Rapid kills of multiple replicas; simultaneous drawing from 3 tabs; show system surviving; show canvas identical across all tabs after chaos
8:30–9:30	Observability	All	Show structured logs (docker logs replica1); show prometheus metrics (curl); show dashboard term/log counters; optionally show TUI
9:30–10:00	Summary	Shrish	Architecture diagram; team member contributions; key technical decisions

17. What Makes This Implementation Stand Out

Feature	What most teams do	What this team does
RPC protocol	HTTP/JSON between replicas	gRPC with .proto file — strongly typed, generated stubs, faster
Log persistence	In-memory only (log lost on restart)	Write-ahead log on disk — survives restarts, restores on boot
Frontend	Plain HTML canvas	React + Vanilla JS hybrid, per-user colour identity, smooth optimistic rendering
Cluster visibility	None	Live embedded dashboard showing elections in real time — term numbers increment on screen
Failure injection	Manual docker stop in terminal	One-click Chaos Mode button with graceful vs hard kill modes
Undo/Redo	Not implemented	Log-compensation undo — appends UNDO_COMPENSATION entries through full RAFT pipeline

Feature	What most teams do	What this team does
Observability	fmt.Println	zap structured JSON logs + Prometheus /metrics + bind-mounted log files + optional TUI
Demo quality	Static terminal logs	Live dashboard + chaos button + multi-tab drawing makes demo cinematic and self-explanatory

Final note to the team

The PRD is the contract. If something is not in this document, discuss it before implementing. If something conflicts with this document, update the document first, then implement. The most common reason distributed system projects fail is undiscussed interface mismatches discovered in Week 3. This PRD exists to prevent that.